Ders 3 Genişletilmiş Notu: typedef, İfadeler ve Operatörler (Kalan, Bit, Koşul), Kontrol Akışı, switch ve Döngüler, Makrolar ve Koşullu Derleme

Ders 3 Genişletilmiş Notu: typedef, İfadeler ve Operatörler (Kalan, Bit, Koşul), Kontrol Akışı, switch ve Döngüler, Makrolar ve Koşullu Derleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tip Tanımlama: typedef
- 2. İfadeler (Expressions)
- 3. Virgül Operatörü
- 4. Operatörlerin Sınıflandırılması ve Kalan Operatörü
- 5. Aritmetik Atama Operatörleri
- 6. Arttırma ve Azaltma Operatörleri
- 7. for Döngüsünün İşleyişi
- 8. İlişkisel Operatörler ve Doğruluk Değeri
- 9. Mantıksal Operatörler ve Kısa Devre
- 10. Bit İşlemleri
- 11. Koşul Operatörü
- 12. Bellek Operatörleri
- 13. Kontrol Akışı: if ve İç İçe if
- 14. switch
- 15. while ve do-while
- 16. Boş İfadeler ve Sık Yapılan Hatalar
- 17. İç İçe Döngüler
- 18. break, continue ve goto
- 19. Ön İşlemci: Makrolar
- 20. Makro ile Fonksiyonun Karşılaştırılması
- 21. Makroyu Kaldırmak ve Hazır Makrolar
- 22. Koşullu Derleme

1. Tip Tanımlama: typedef

Geçen derste numaralandırma (enumeration) ve `void` veri tipi görülmüştü. Ders, kalan konu olan tip tanımlamayla (type definition) devam etti.

`typedef` çok basit bir araçtır: Var olan bir C tipini kendi seçtiğiniz bir isimle yeniden adlandırır. Hoca bunu önce `#define` ile karşılaştırdı. `#define MAX 10` yazdığımızda ön işlemci derleme aşamasında `MAX` gördüğü her yere 10 yazar. `typedef` de benzer biçimde bir C tipini verir ve bu tipin kod içinde başka bir isimle geçeceğini söyler.

```
typedef long int int32;
int32 j;              /* long int j; yazmak yerine */
```

Hoca iki farkı belirtti: `typedef` tam olarak bir ön işlemci komutu gibi değildir, bu yüzden `#` ile başlamaz ve sonuna noktalı virgül konur.

> **Ders sonrası not:** Hoca başta `typedef`'i "bir ön işlemci yer değiştirmesi" diye tanıttı, hemen ardından "tam bir ön işlemci gibi değil" dedi. Doğrusu ikincisidir. `gcc -E` ile denendi: `#define MAX 10` ön işlemciden sonra kayboldu ve yerine 10 geldi, `typedef` satırı ise olduğu gibi kaldı. Yani `typedef` metin yer değiştirmesi değildir, büyük olasılıkla derleyicinin kendisi tarafından işlenir.

> **Ek bilgi (derste anlatılmadı):** `int32` adı `long int` tipinin 32 bit olduğunu düşündürse de bu sabit değildir. Bu notun hazırlandığı Linux makinede `gcc` ile `sizeof(long)` 8 bayt çıktı. Tiplerin genişliği büyük olasılıkla derleyiciye ve platforma göre değişir, bu yüzden isim yalnız bir etikettir.

Kullanımını göstermek için hoca derste kısa bir deneme yaptı: `int` tipine `tamsayi` adını verdi ve aşağıda `tamsayi x, y;` diye değişken tanımlayıp programı derledi.

```
typedef int tamsayi;
tamsayi x, y;
```

1.1 Asıl Kullanım Alanı: Yapılar (struct)

`typedef`'i asıl yapılarla (structure) çok kullanacağız. Yapı, polinom veya karmaşık sayı gibi kendi veri tiplerimizi tasarlamakta kullandığımız bir araçtır. Hoca karmaşık sayı örneğini kurdu: Bir karmaşık sayının `float` tipinde bir gerçel kısmı ve bir sanal kısmı vardır.

```
struct complexNumber {
    float real;
    float imaginary;
};
```

Bu tipten bir değişken tanımlarken her seferinde önüne `struct` kelimesini yazmamız gerekir: `struct complexNumber c1;`. Bu uzun bir yazımdır ve program yazmayı güçleştirir. Fonksiyonlarda da aynı uzunluk tekrar eder: `c1`'i bir `foo` fonksiyonuna göndermek istiyorsak, parametre listesine de `struct complexNumber c` yazmak gerekir.

Bunu önlemek için tanımın başına `typedef` eklenir. Hoca burada sınıfa bir soru sordu: `typedef int tamsayi;` yazarken C tipinden sonra bir boşluk koyup yeni ismi yazıyorduk. Yapı tanımında bu boşluk nereye gelir? Yanıt için C tipinin nerede başlayıp nerede bittiğine bakmak gerekir. Tip `struct` kelimesiyle başlar ve kapanan süslü paranteze kadar gider. Yeni isim bu parantezden sonra, noktalı virgülden önce yazılır:

```
typedef struct complexNumber {
    float real;
    float imaginary;
} complex;

void foo(complex c1, complex c2);
```

Artık `struct complexNumber c2;` yerine `complex c2;` yazabiliriz. Hoca ikinci bir yazım biçimi de gösterdi: Önce yapıyı `typedef` kullanmadan tanımlamak, ardından ayrı bir satırda ona isim vermek. Bu sefer yeni ismi büyük harfle yazdı. İki yazım da aynı işi görür.

```
struct complexNumber {
    float real;
    float imaginary;
};
typedef struct complexNumber COMPLEX;
```

1.2 Prototipin Yeri

Bir yapıyı parametre alan fonksiyonun prototipini yazarken sıra önemlidir. Hoca prototipi yapı tanımının üstüne yazıp "olur mu?" diye sordu ve cevabı kendisi verdi: Olmaz, çünkü o noktada yapı henüz tanımlanmamıştır.

> **Hocanın vurgusu:** Önce yapı tanımlanır, sonra onu kullanan fonksiyonun prototipi yazılır.

`gcc` ile denendi: `complex` tipini kullanan prototip `typedef`'ten önce yazılınca "unknown type name" hatası verdi, sonra yazılınca derlendi.

Hoca aynı noktayı biraz sonra başka bir denemede de gördü: Derleyici hata verince karmaşık sayı tanımını yukarı taşıyarak hatayı giderdi. Fonksiyonun gövdesi henüz boşken derlemede görülen iletilerin ise uyarı (warning) olduğunu, programın çalışmasını engellemediğini söyledi.

Hocaya göre `typedef`'in en yaygın kullanımı budur. `typedef int tamsayi;` gibi kullanımlar pek tercih edilmez, kimse sizden C dilini Türkçeleştirmenizi beklemez. Yapı tanımlarında ise kodu daha kısa ve daha anlaşılır hale getirir. `typedef` konusuna sonraki derslerde yapılarla birlikte tekrar dönülecek.

2. İfadeler (Expressions)

Hoca bu kısmı hızlı bir tekrar olarak anlattı. İfadeler dört gruba ayrılır: sabit ifadeler, tam sayı ifadeleri, ondalıklı (floating point) ifadeler ve işaretçi (pointer) ifadeleri.

2.1 Sabit İfadeler

Sabit ifadeler yalnız sabit değerlerden oluşur. Kodun içine durduk yere `5;` yazmak bile geçerli bir ifadedir, herhangi bir soruna yol açmaz, sadece bir şey yapmaz.

Slayttaki örnek `5 + 6 * 13 / 3.0` idi. Hoca işlem sırasını sınıfla birlikte kurdu:

- Çarpma ve bölme aynı önceliktedir ve soldan sağa yapılır, toplama en sona kalır.
- $6 \cdot 13 = 78$.
- $78/3.0 = 26.0$. `3.0` ondalıklı olduğu için sonuç da ondalıklıdır.
- $5 + 26.0 = 31.0$.

Sonuç python ile hesaplandı: 31.0.

2.2 Tam Sayı İfadeleri

Bu ifadeler değişken kullanılarak kurulur (kısa nottaki slayt örneklerine göre `int j, k` için): `j`, `j / k * 3`, `k - 'a'`, `3 + (int)5.0`. Hepsi geçerlidir. Tek başına bir değişken ismi yazmak da ifadedir. Veriler birbirine dönüşebiliyorsa iki tarafta aynı türün olması gerekmez.

`k - 'a'` çalışır mı sorusunun cevabı evettir, çünkü karakterin sayısal bir karşılığı vardır. `'a'` 97'ye karşılık gelir, yani ifade `k`'daki değerden 97 çıkarır. Hoca `char` tipini "aslında 8 bitlik, 0'dan 255'e kadar değer alan bir tam sayı" olarak anlattı. Sadece bu sayıları kullandığımız sembollere atamışızdır.

> **Ders sonrası not:** Hoca `char` için 0-255 aralığını söyledi. Bu aralık `unsigned char` içindir. Bu notun hazırlandığı makinede `gcc` ile `CHAR_MIN` -128, `CHAR_MAX` 127 çıktı, yani düz `char` işaretli davrandı. `char`'ın işaretli olup olmaması büyük olasılıkla derleyiciye bağlıdır. Tam sayı olduğu bilgisi ise doğrudur.

`3 + (int)5.0` için önce tip dönüşümü, sonra toplama yapılır: $3 + 5 = 8$ (python ile hesaplandı).

2.3 Ondalıklı İfadeler

Bir operatörün sol veya sağ tarafındaki operandlardan en az biri ondalıklıysa ifade ondalıklı olarak hesaplanır. `3 + (float)4` ifadesinde akla şu soru gelir: Önce toplama mı, önce tip dönüşümü mü? Tip dönüşümü (type casting) her zaman daha önce yapılır. `(float)4` 4.0 olur, 3 ile toplanınca 7.0 elde edilir.

2.4 İşaretçi İfadeleri

`p` bir adresi gösteren işaretçiyse `p` yazmak bir ifadedir. `p + 1` dediğimizde işaretçinin türü devreye girer. Hoca önce "`p + 1 * sizeof(p)`" dedi, sonra kendisi düzeltti: Doğrusu `p + 1 * sizeof(int)`, yani işaretçinin gösterdiği türün boyutudur. `int` 4 bayt ise adres 4 ilerler. `gcc` ile

denendi: `int *p` için `p + 1` ile `p` arasındaki fark 4 bayt çıktı. Kısa nota göre slaytta `"abc"` gibi bir karakter dizisi sabiti de bu gruptadır, çünkü elemanlarına işaretçi üzerinden erişilebilir.

Öncelik (precedence) ve birleşme yönü (associativity) tablosu daha önce anlatıldığı için geçildi, ayrıntılı tablo ders notlarındadır.

3. Virgül Operatörü

Virgülde işlem soldan sağa yapılır. Hoca `return 3,4,5;` diye bir ifade yazıp karşılığını bir yerde alsak hangi değerin geleceğini sordu.

Soru: `return 3,4,5;` hangi değeri döndürür? **Cevap:** 5. Önce 3'e, sonra 4'e, sonra 5'e bakılır ve geriye sonuncusu döner.

Hoca bunu `x = 3,4,5` yazarak denemek istedi, ama değişken tanımlı olmadığı için derleyici hata verdi. Bunun üzerine bir fonksiyon yazdı:

```
int f(void) {
    return 2, 3, 4;
}
```

Fonksiyonun çağrısından en sondaki değer, yani 4 geldi. Tek başına `2, 3, 4;` diye bir satır yazmak da derlenir ama bir anlam taşımaz.

> **Ders sonrası not:** Hoca `return 3,4,5` ile `x = 3,4,5` yazımının aynı sonucu vereceğini söyledi, ikincisi derste çalıştırılamadı. `gcc` ile derlenip çalıştırıldı: `x = 3,4,5;` satırında `x` 3 olur, çünkü atama virgülden daha önceliklidir. `x` 5 olsun isteniyorsa `x = (3,4,5);` yazmak gerekir. `gcc -Wall` bu satırlarda etkisiz operand uyarısı verir.

4. Operatörlerin Sınıflandırılması ve Kalan Operatörü

Parantezlerin işlem önceliğindeki rolü biliniyor. Operatörler farklı açılardan sınıflandırılabilir. Örneğin işlevlerine göre matematiksel, mantıksal gibi gruplar yapılabilir. Bir başka sınıflandırma operand sayısına göredir:

- **Tekli (unary):** Tek operand alır.
- **İkili (binary):** İki operand alır, biri soluna biri sağına gelir. Toplama, çıkarma, karşılaştırma ve kaydırma işlemleri böyledir. Hocaya göre kullandığımız operatörlerin yüzde 70-80'i bu gruptadır.
- **Üçlü (ternary):** Üç operand alır. Bu, birazdan görülecek olan koşul operatörüdür (`?:`).

4.1 Kalan Operatörü (%)

Kalan (remainder) operatörünün diğer aritmetik operatörlerden bir farkı vardır. Toplamanın iki tarafına tam sayı da ondalıklı sayı da yazabilirsiniz. Kalan operatörü ise yalnız tam sayı operand alır. `6 % 2` geçerlidir, `6.0 % 2.0` geçersizdir. `gcc` ile denendi: `6.0 % 2.0` derleme hatası verdi, `6 % 2` 0 çıktı.

Hoca, ANSI standardında kalan ile bölme arasında şu ilişkinin bulunduğunu söyledi. `a` ve `b` tam sayıları için:

$$a = (a/b) \cdot b + a \bmod b$$

C ile yazılışı `a == (a / b) * b + a % b` şeklindedir ve buradaki bölme tam sayı bölmesidir.

Hoca operandlar negatifse kalanın negatif de pozitif de olabileceğini, bunun gerçekleştirime (implementation) göre değişebileceğini söyledi.

> **Ders sonrası not:** Hoca kalanın işaretinin gerçekleştirime bağlı olduğunu söyledi. Bu büyük olasılıkla C89 için geçerliydi. C99'dan beri büyük olasılıkla tam sayı bölmesi sıfıra doğru kesilir ve kalan bölünenin işaretini alır. `gcc` ile derlenip çalıştırıldı: `-7 / 3` = -2, `-7 % 3` = -1, `7 % -3` = 1. Yukarıdaki ilişki de `a = -7`, `b = 3` için doğru çıktı.

5. Aritmetik Atama Operatörleri

Atama operatörleri de ikilidir. Hoca slayttaki `m += n + x - y` ifadesinin açılımını sordu. Değeri değil, açılımı isteniyordu.

Soru: `m += n + x - y` neye açılır? **Cevap:** Sağ taraf bütünüyle paranteze alınır: `m = m + (n + x - y)`. Operatör bunu arka planda kendisi yapar. Önce parantez içi hesaplanır, sonra `m`'nin mevcut değeri üzerine eklenir ve `m`'nin yeni değeri elde edilir.

Slayttaki değerler `int m = 3, n = 4; float x = 2.5, y = 1.0;` idi. Adım adım:

- $n + x = 4 + 2.5 = 6.5$.
- $6.5 - y = 6.5 - 1.0 = 5.5$.
- $m + 5.5 = 3 + 5.5 = 8.5$.
- `m` `int` olarak tanımlandığı için 8.5 değeri `int`'e dönüştürülür. Bu dönüşümde virgülden sonraki kısım atılır, yani yuvarlama yapılmaz.

Sonuç: `m` = 8 (python ile hesaplandı, `gcc` ile de aynı çıktı).

Soru: `m`'nin yeni değeri 8.5 mi, 9 mu olur? **Cevap:** 8. Değişken `int` olduğu için ondalık kısım atılır.

Hoca diğer satırları aynı yöntemle değerlendirmeyi öğrencilere bıraktı. Kısa nota göre slaytta `m /= x * n + y`, `n %= y + m` ve `x += y -= m` satırları vardı. Aynı başlangıç değerleriyle bağımsız olarak hesaplanınca:

- `m /= x * n + y`: $x \cdot n + y = 11.0$, $3/11.0 \approx 0.27$, `int`'e dönünce `m` = 0 (python ile hesaplandı).
- `x += y -= m`: Atamalar sağdan sola çalışır. $y = 1.0 - 3 = -2.0$, sonra $x = 2.5 + (-2.0) = 0.5$ (python ile hesaplandı).

  Ders sonrası not: Kısa nottaki `n %= y + m` satırı bu değişken tipleriyle derlenmez. `y` `float` olduğu için sağ taraf ondalıklıdır ve `%` yalnız tam sayı operand alır. `gcc` ile denendi: "invalid operands to binary %" hatası verdi. Slaytta değişken tipleri farklı olabilir.

6. Arttırma ve Azaltma Operatörleri

Sonek (postfix) ve önek (prefix) arttırma, ikisi de değişkeni 1 arttırır. Fark, ifadenin o anda hangi değeri verdiğindedir.

Sonek arttırmada önce değişkenin mevcut değeri elde edilir, ikinci aşamada değişken 1 arttırılır. `a = 0`, `b = 1` iken:

```
a = b++;     /* a = 1, b = 2 */
```

Önek arttırmada önce değişken arttırılır. `b` 2 olur, bu değer atama ile `a`'ya geçer. Aynı başlangıç değerleriyle:

```
a = ++b;     /* a = 2, b = 2 */
```

İki sonuç da `gcc` ile doğrulandı. Hoca sonek arttırmanın arka planını şöyle anlattı: `t` gibi geçici bir değişkene `b` atanır, sonra `b` bir arttırılır. Böylece `t`'de eski değer, `b`'de güncel değer bulunur.

```
t = b;
b = b + 1;
```

Bir `for` döngüsünün `x++` kısmında olduğu gibi atama yapılmıyorsa iki yazımın sonucu aynıdır.

> **Ders sonrası not:** Hoca önce sonek arttırmanın iki aşamalı olduğu için biraz gecikmeli olduğunu söyledi, sonra "Assembly seviyesinde `x++` daha hızlı çalışır" dedi. Altyazıdaki bu iki ifade birbirini tutmuyor. `gcc` ile `-O0` ve `-O2` seçenekleriyle denendi: Sonucu kullanılmayan `x++` ve `++x` iki durumda da aynı Assembly kodunu üretti. Döngü arttırımında bu makinede hız farkı yoktur.

6.1 Slayttaki printf Örneği

Slaytta `j` ve `k` 5 iken iki `printf` çifti vardı. Hoca ilk satırda ikisinin de 5 görüneceğini söyledi, çünkü `printf`'e argüman vermek bir anlamda atama gibi davranır: `a = j++, b = k--` yazmışız gibi eski değerler geçer. Sonraki satırda arttırılmış ve azaltılmış değerler, 6 ve 4 görülür. Önekli biçimde ise iki satırda da 6 ve 4 görülür.

Slayttaki kodun anlatıma göre yeniden yazımı aşağıdadır. `gcc -Wall` ile derlendi, uyarı vermedi.

```c
#include <stdio.h>

int main(void) {
    int j = 5, k = 5;
    printf("%d %d\n", j++, k--);   /* 5 5 */
    printf("%d %d\n", j, k);       /* 6 4 */

    j = 5;
    k = 5;
    printf("%d %d\n", ++j, --k);   /* 6 4 */
    printf("%d %d\n", j, k);       /* 6 4 */
    return 0;
}
```

6.2 Öncelikle Birlikte Kullanım

Hoca bu tür ifadelerde hem işlem önceliğine hem de operatörün önek veya sonek olmasına birlikte dikkat etmek gerektiğini söyledi. Slayttaki başlangıç değerleri `int j = 0, m = 1, n = -1, s;` idi. Satırlar birbirinden bağımsızdır, yani her biri aynı başlangıç değerleriyle değerlendirilir.

`s = m++ - --j;` satırı:

- `m++` çıkarmanın sol operandıdır ve mevcut değeri, yani 1'i verir. `m` daha sonra 2 olur.
- `--j` çıkarmadan önce azaltılır, -1 verir.
- $s = 1 - (-1) = 2$.

`s = m += ++j * 2;` satırı:

- Çarpma için önce `++j` hesaplanır: `j` 1 olur.
- $1 \cdot 2 = 2$.
- `m`'nin mevcut değeri 1'dir: $m = 1 + 2 = 3$.
- `s` de 3 olur.

İki sonuç da `gcc` ile doğrulandı.

Hoca son olarak `s = m++ * m++;` satırını gösterdi. Bu satırın sonucu nereden başlanarak hesaplandığına göre değişir. Hoca bunu gerçekleştirime bağlı diye nitelendirdi ve bu tür ifadelerin üzerinde durulmayacağını söyledi.

> **Ders sonrası not:** Hoca bu satır için "gerçekleştirime bağlı" dedi. Standarda göre durum büyük olasılıkla daha ağırdır: Aynı değişken arada bir sıra noktası olmadan iki kez değiştirildiği için tanımsız davranıştır (undefined behavior). `gcc -Wall` bu satırda `-Wsequence-point` uyarısı verdi. Böyle ifadelerin yazılmaması gerekir.

7. for Döngüsünün İşleyişi

Virgül operatörü özellikle birden fazla değişkenli `for` döngülerinde karşımıza çıkar. Bu örnekten önce hoca, `for` döngüsüyle ilgili yaygın bir yanılgı için hazırladığı bir slaytı gösterdi. Slayttaki kod Java'dır, ama C'de işleyiş aynıdır. Döngü `for(i = 0; i < 2; i++)` biçimindeydi. Çalışma sırası şöyledir:

1. Başlangıç ataması yapılır. Sayaç üst sınırı 2 değil 20 olsa bile bu atama yalnız bir kez yapılır.
2. Koşul kontrol edilir. Doğruysa döngünün içine girilir.

3. Gövdedeki bütün ifadeler çalışır.
4. Arttırma kısmı (`i++`) çalışır.
5. 2. adıma dönülür: Döngünün devam edip etmeyeceği yeniden kontrol edilir.
6. Koşul yanlışsa döngüden çıkılır ve bir sonraki satırdan devam edilir.

Örnekte `i` 2 olduğunda `i < 2` sağlanmaz ve döngü biter. Döngüden sonraki satıra gelindiğinde `i`'nin değeri 2'dir.

> **Hocanın vurgusu:** Döngüdeki sayaç değişkeni aşağıda bir yerde kullanılacaksa, son değerinin 2 olarak kaldığını hesaba katmak gerekir. Bu çok sık yapılan hatalardan biridir.
>
> **Ders sonrası not:** Hoca sayacı döngünün içinde tanımlamakla (`for(int i = 0; ...)`) dışarıda tanımlamanın fark etmediğini söyledi. Çalışma sırası için bu doğrudur, ama döngüden sonra sayacı kullanacaksanız fark eder. `gcc` ile denendi: `for(int i = 0; i < 2; i++)` sonrasında `i` kullanılınca "undeclared" hatası verdi. Sayacın son değeri gerekiyorsa `i` döngüden önce tanımlanmalıdır.

7.1 İki Değişkenli Döngü

Hoca slayttaki iki değişkenli döngüyü, 100 yerine 10 alarak sınıfa sordu:

```
for (j = 0, k = 10; k - j > 0; j++, k--)
    printf(...);
```

Önce `j`'ye 0, sonra `k`'ya 10 atanır ve bu yalnız bir kez yapılır. Sonra koşula bakılır, gövde çalışır, ardından `j` bir arttırılır, `k` bir azaltılır, tekrar koşula bakılır.

Soru: Bu döngü kaç kez döner? Sınıftan "10 kere" cevabı geldi. **Cevap:** 5 kez, çünkü hem `j` artıyor hem `k` azalıyor. Hoca döngüyü adım adım izledi:

Adım	`j`	`k`	`k - j`	`k - j > 0`	Sonuç
1	0	10	10	doğru	1. yazdırma
2	1	9	8	doğru	2. yazdırma
3	2	8	6	doğru	3. yazdırma
4	3	7	4	doğru	4. yazdırma
5	4	6	2	doğru	5. yazdırma
6	5	5	0	yanlış	döngüden çıkış

Döngü 5 kez döner ve sonunda `j` = 5, `k` = 5 olur. Tablo `gcc` ile çalıştırılan kodla doğrulandı.

> **Ders sonrası not:** Hoca izleme sırasında "7 eksi 3 sıfırdan küçük mü? Değil" diye okudu ve "5 eksi 5 sıfırdan küçük mü? Değil. O zaman çıktık" dedi. Koşul `k - j > 0` olduğu için doğru okuma "sıfırdan büyük mü?" şeklindedir. Son adımda fark 0'dır, 0'dan büyük olmadığı için döngüden çıkılır.

7.2 Karşılaştırma Kaç Kez Yapılır?

Hoca bunun algoritma analizinde çok önemli bir soru olduğunu söyledi ve `for(i = 0; i < 5; i++)` döngüsünü ele aldı. `i` 0, 1, 2, 3, 4 iken karşılaştırma yapılır ve gövde çalışır. `i` 5 olduğunda da karşılaştırma yapılmak zorundadır, ama bu kez gövdeye girilmez.

- Gövde 5 kez çalışır.
- Karşılaştırma 6 kez yapılır.

Başlangıç 1 olsaydı (`for(i = 1; i < 5; i++)`) gövde 4 kez, karşılaştırma 5 kez olurdu. İki durum da `gcc` ile sayılarak doğrulandı.

> **Hocanın vurgusu:** Zaman karmaşıklığı ve bellek karmaşıklığı analizlerinde döngülerin tekrar sayısı önemlidir. `for` döngüsünde tekrar sayısı genellikle bu karşılaştırma işlemi üzerinden hesaplanır.

8. İlişkisel Operatörler ve Doğruluk Değeri

Karşılaştırma operatörlerinin diğer operatörlerden farkı, geriye doğru ya da yanlış bilgisi döndürmeleridir. C'de ayrıca bir doğru-yanlış değeri kullanılmaz: Doğru yerine 1, yanlış yerine 0 döner.

Bir detay daha vardır: Sıfır dışındaki bütün sayısal değerler doğru kabul edilir. Hoca bunu derste denedi. `if` koşuluna negatif bir sayı yazıp içine bir yazdırma koydu ve yazdırma çalıştı. Aynı yere `while` yazınca sonsuz döngü oluştu ve ekrana sürekli -1 yazıldı.

```
if (-5)
    printf("-1");     /* çalışır */

while (-5)
    printf("-1");     /* sonsuz döngü */
```

8.1 Ondalıklı Sayıları == ile Karşılaştırmak

Hoca sayısal analizde en çok yapılan hatalardan birini anlattı. Bir beklenen değer (örneğin 5.0) ve bir hesaplanan değer (örneğin 4.0) olsun, ikisi de `double`. Amaç, ikisi arasındaki farkın mutlak değeri 0.001 olana kadar döngüyü çevirmek olsun:

```
double beklenen = 5.0, hesaplanan = 4.0;
while (fabs(beklenen - hesaplanan) == 0.001) {
    /* ... */
}
```

Hocanın uyarısı şudur: Bu fark hiçbir zaman tam olarak 0.001'e eşit olmayabilir, 0.001'e yakın bir değer olur ve sonsuz döngüye girersiniz. Bunun yerine fark bir epsilon değerinin altına düşene kadar döngü sürdürülür. Böylece daha doğru bir sonuç elde edilir.

> **Hocanın vurgusu:** Ondalıklı sayılarda == kullanmak yerine bir tolerans (epsilon) ile karşılaştırma yapılır.
>
> **Ders sonrası not:** Uyarı doğrudur, ama koddaki koşul sözle anlatılanla aynı değildir. Hoca "eşit olana kadar dönsün" dedi; yazılan `while (... == 0.001)` ise "eşit olduğu sürece dönsün" demektir. `gcc` ile denendi: 5.0 ve 4.0 için bu koşul 0 çıktı, yani döngüye hiç girilmez. Sonsuz döngü riski `!=` yazımındadır: `while (fabs(beklenen - hesaplanan) != 0.001)`. Doğru biçim `while (fabs(beklenen - hesaplanan) >= epsilon)` şeklindedir.
>
> **Ders sonrası not:** Hoca "== operatörü bir anlamda çıkarma yapıyor" diye açıkladı. Asıl neden büyük olasılıkla ondalıklı sayıların ikilik tabanda çoğu zaman tam gösterilememesidir. Hesaplanan iki değer neredeyse eşit olsa bile bitleri farklı olabilir, bu yüzden == yanlış verir.

8.2 Birleşme Yönü Örneği

Kısa nottaki slayt değerleri `int j = 0, m = 1, n = -1; float x = 2.5, y = 0.0;` idi. Hoca `j <= m >= n` ifadesini ele aldı. İlişkisel operatörlerde birleşme yönü soldan sağadır. Hoca önce "sağdan sola" diye başladı, sonra tablodan kontrol edip soldan sağa olduğunu söyledi.

- `j <= m`: $0 \leq 1$ doğrudur, sonuç 1 olur. Buraya "true" değil 1 yazılır.
- `1 >= n`: $1 \geq -1$ doğrudur, sonuç 1 olur.

İfadenin sonucu 1'dir. Hoca ifadenin başına `4 *` eklenseydi doğru değerin aritmetiğe 1 olarak katılacağını, 4 ile doğruyu çarpmanın aslında $4 \cdot 1$ yapmak olduğunu ekledi. `gcc` ile denendi: `4 * (j <= m)` 4 değerini verir, ifadenin tamamı yine 1 çıkar.

9. Mantıksal Operatörler ve Kısa Devre

Mantıksal operatörler iki veya daha fazla mantıksal ifadeyi birleştirir. Yani bir `if`, `while` ya da `for` içinde birden çok şart sunmak için kullanılır. Örneğin `x < y` ifadesini A, `x < z` ifadesini B olarak düşünelim:

- `A && B`: İkisi de doğruysa 1, diğer durumlarda 0.
- `A || B`: En az biri doğruysa 1.

Bu tür işlemlerde kısa devre (short circuit) denilen bir kolaylık vardır. Önce sol taraf değerlendirilir ve sonuç belli olmuşsa sağ tarafa hiç bakılmaz.

Soru: `0 && (...)` ifadesinin sonucu nedir? **Cevap:** 0. VE işleminin sol tarafı 0 olduğu için sağ tarafa bakmaya gerek yoktur. Hocanın benzetmesiyle, boş kümenin başka bir kümeyle kesişimi yine boş kümedir.

Soru: `1 || (...)` ifadesinin sonucu nedir? **Cevap:** Her zaman 1. Sol taraf 1 ise sağa bakılmaz.

Soru: `1 && (x < y)` ifadesinin sonucu nedir? **Cevap:** `x < y`'nin sonucu neyse odur. Doğruysa 1, yanlışsa 0.

Mantıksal değilleme (`!`) bir mantıksal ifadenin sonucunu tersine çevirir: Doğruyu yanlış, yanlışı doğru yapar. Slayttaki diğer satırların çözümleri ders notlarında bulunduğu için geçildi.

10. Bit İşlemleri

Bit seviyesinde VE, VEYA, özel VEYA (XOR), tümleyen (complement) ve kaydırma (shift) işlemleri vardır. Örnekler 16 bitlik gösterimle anlatıldı.

10.1 Kaydırma

Sola kaydırma, `5 << 1`. Her bit bir sola kayar. Sağ taraftan 0 ile besleme yapılır. Soldan taşan bit dışarı gider, dönüp başa gelmez. Bu döndürme (rotation) değildir. Sonuç bir 2 ile çarpma gibidir.

```
5       = 0000 0000 0000 0101
5 << 1  = 0000 0000 0000 1010  = 10
```

Sağa kaydırma, `255 >> 3`.

```
255       = 0000 0000 1111 1111
255 >> 3  = 0000 0000 0001 1111  = 31
```

Hoca bunu 2'ye art arda bölmeye benzetti ve yaklaşık değerler olarak 128, 64, 32 dedi.

> **Ders sonrası not:** Hocanın söylediği 128, 64, 32 yaklaşık değerlerdir. Sağa kaydırmada kesin ara değerler 127, 63, 31'dir (python ile hesaplandı), çünkü her adımda sağdan düşen bit atılır. $255/8 = 31.875$ olduğundan `255 >> 3` 8'e tam sayı bölmesi gibi davranır ve 31 verir.

`8 << 10`. 8 aslında 2^3'tür. 10 kez sola kaydırılınca 2^{13} = 8192 elde edilir (python ile hesaplandı).

```
8        = 0000 0000 0000 1000
8 << 10  = 0010 0000 0000 0000  = 8192
```

`1 << 15`. 16 bitlik gösterimde 1, 15 kez kaydırılınca en soldaki bite gelir. Hoca sayı işaretliyse bunun işaret biti etkisi yaptığını belirtti.

```
1 << 15  = 1000 0000 0000 0000
```

Bu bit dizisi işaretsiz yorumlanırsa 32768, 16 bit işaretli yorumlanırsa -2^{15} = -32768 eder (python ile hesaplandı).

10.2 VE ve VEYA

Örnekte iki onluk sayı vardır: 9430 ve 5722. Bit işlemi için sayıları ikilik tabana çevirmek gerekir, bunun kolay yolu önce onaltılığa geçmektir. Onaltılıkta 10 A'ya, 11 B'ye, 12 C'ye, 13 D'ye karşılık gelir. Hoca rakamları tek tek açtı.

```
9430 = 0x24D6 = 0010 0100 1101 0110
5722 = 0x165A = 0001 0110 0101 1010
```

VE (&): Karşılıklı bitler ikisi de 1 ise sonuç 1, diğer durumlarda 0 olur.

```
  0010 0100 1101 0110
& 0001 0110 0101 1010
--------------------
  0000 0100 0101 0010  = 0x0452
```

Soru: `0x452` onluğa nasıl çevrilir? **Cevap:** Rakamlar sağdan sola doğru artan kuvvetlerle 16'nın kuvvetleriyle çarpılır:

$$2 \cdot 16^0 + 5 \cdot 16^1 + 4 \cdot 16^2 = 2 + 80 + 1024 = 1106$$

VEYA (|): Karşılıklı bitlerden en az biri 1 ise sonuç 1, ikisi de 0 ise 0 olur.

```
  0010 0100 1101 0110
| 0001 0110 0101 1010
--------------------
  0011 0110 1101 1110  = 0x36DE = 14046
```

İki sonuç python ile hesaplandı ve `gcc` ile de aynı çıktı.

10.3 XOR ve Tümleyen

XOR'un durumu biraz farklıdır. A ve B gibi iki bitin dört farklı birleşimi vardır:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Girdiler aynıysa sonuç 0, farklıysa 1'dir. Hoca XOR'un şifrelemede ve kriptolojide çok kullanıldığını, anahtarla veriyi ilişkilendirmede ondan faydalanıldığını söyledi.

Bit düzeyinde değilleme (~) ise 0'ları 1, 1'leri 0 yapar. Bu operatörlerin atamalı biçimleri de vardır, toplamadaki += ile aynı mantıkla çalışırlar.

11. Koşul Operatörü

Koşul operatörü, basit bir `if` deyimini daha kısa yazmanın aracıdır. Önce doğru ya da yanlış sonuç veren bir koşul yazılır, sonra soru işareti gelir. Koşul doğruysa ilk değer, yanlışsa ikinci değer verilir. `A` ve `B` tek başına durmaz, sonuç bir değişkene atanır:

```
x = koşul ? A : B;
```

Slayt örneği `z = (x < y) ? x : y;` idi. `x`, `y`'den küçükse `z`'ye `x`, değilse `y` atanır.

Soru: Burada ihmal edilen durum nedir? **Cevap:** Eşitlik. `x`, `y`'ye eşitse hangi dalın çalışacağı ayrıca düşünülmemiştir.

Bunu ele almak için operatör iç içe kullanılabilir. Hoca dış koşulun doğru kolunu ve yanlış kolunu gösterdi. Yanlış kolunun içine ikinci bir koşul operatörü yazılır ve onun da kendi doğru ve yanlış kolları olur:

```
v = (x < y) ? x : ((x == y) ? 1 : y);
```

Burada `x`, `y`'den küçükse `x`, eşitse 1, büyükse `y` döner. Satır `gcc -Wall` ile derlendi.

12. Bellek Operatörleri

Dizinin istenen elemanına erişim zaten biliniyor. Yapı üyelerine erişim operatörleri, işaretçili ve işaretçisiz biçimleriyle, zamanı gelince konuşulacak. Hoca burada * ve & üzerinde durdu ve bu sembollerin daha önce başka anlamlarda da görüldüğünü hatırlattı: * çarpma, & bit düzeyinde VE'dir.

Soru: Adres alan `&` ve gösterilen değeri veren `*`, çarpmadan ve bit VE'den nasıl ayrılır? **Cevap:** Operand sayısından. Bir sınıf cevabı "bir ifadenin başına gelirse" şeklindeydi ve hoca bunu açtı: Çarpma iki operand ister (`a * b`). Tek operandla `*a` yazılırsa `a` adresinin gösterdiği değer anlamına gelir. Aynı şekilde bit VE iki operand alır. Tek operandla `&x` yazılırsa `x`'in adresi anlamına gelir. `scanf`'te bu yüzden `&` kullanıyoruz.

13. Kontrol Akışı: if ve İç İçe if

`if` zaten çok görülen bir konudur, ama dikkat edilecek bir nokta vardır. Şarta bağlı tek bir ifade çalışacaksa süslü parantez konmayabilir. Fakat alt alta yazmak yanıltıcıdır:

```
if (x != 0)
    statement1;
    statement2;
```

Burada `x` 0'dan farklıysa `statement1` çalışır. `statement2` ise `x` ne olursa olsun çalışır, çünkü şarta bağlı değildir. Girinti bunu değiştirmez.

> **Hocanın vurgusu:** Tek satır bile yazacak olsanız her durumda `if` ve `else` bloklarını süslü parantezlerle açıp kapatın. "Tutarsız sonuç alıyorum" şikayetlerinin nedenlerinden biri budur.

13.1 İç İçe if ve Boşta Kalan else

İç içe `if` kullanılabilir, ama boşta kalan `else` (dangling else) durumuna dikkat etmek gerekir. Kısa nota göre slayttaki kod şudur:

```
if (a < b)
    if (a < c)
        return a;
    else
        return c;
else if (b < c)
    return b;
else
    return c;
```

Hoca bloğu şöyle okudu: `a < b` iken `a < c` ise `a` döner, değilse `c`. `a`, `b`'den küçük değilse ikinci bir şart gelir: `b < c` ise `b`, değilse `c` döner. Hoca, her dalda `return` kullanıldığı için bazı `else` dallarının hiç çalışmayacağı bir yapı kurulabileceğini söyledi. Sonra bu kodda sorunlu bir dal olup olmadığını sınıfa sordu, cevap gelmeyince düşünmeyi öğrencilere bıraktı.

> **Ders sonrası not:** Soruya derste cevap verilmedi. Bu kod `gcc` ile derlenip her sayı 0, 1 veya 2 olacak şekilde bütün 27 birleşimle çalıştırıldı: Her durumda üç sayının en küçüğünü döndürdü. Kısa notta da belirtildiği gibi kodda hatalı dal görülmüyor ve `else return c;` satırı en yakın `if`'e bağlanıyor.

14. switch

`switch` çok kullanılmayacak, ama zaman zaman iç içe geçmiş `if`'leri daha düzgün yazmayı sağlar. Parantez içine çok değer alabilen bir ifade yazılır, ardından `case` etiketleri sıralanır. Her `case`'in sonunda `switch`'ten çıkmak için bir `break` gerekir. Hiçbir `case` eşleşmezse en altta yazılan `default` çalışır.

`switch` ifadesi C'de her türlü değeri kabul etmez. Slayttaki örnekte `int` bir değer kullanılıyordu, ama hoca karakterin de çalıştığını gösterdi.

Soru: `switch` karakteri neden kabul ediyor? **Cevap:** Karakter de bir anlamda tam sayıdır. `switch` ASCII değerine, yani sayısal karşılığına bakar. Hoca bunu `char c = 'A';` ve `case 65:` ile gösterdi: `'A'` 65 olduğu için eşleşti.

Hoca sonra `break`'leri kaldırarak ve değeri değiştirerek denemeler yaptı. Denemelerin toplu hali aşağıdadır. Kod anlatıma göre yeniden yazıldı, `gcc -Wall` ile derlendi.

```c
#include <stdio.h>

int main(void) {
    char c = 'A';

    switch (c) {
        case 65:
            printf("A\n");
        case 2:
            printf("iki\n");
        default:
            printf("varsayilan\n");
    }
    return 0;
}
```

`break`'siz durumda eşleşen `case`'ten sonrakiler de çalışır. Çıktılar:

c değeri	break var	break yok
`'A'`	`A`	`A`, `iki`, `varsayilan`
`2`	`iki`	`iki`, `varsayilan`
`22`	`varsayilan`	`varsayilan`

`switch` içinde string kullanılamaz. Hoca iki karakterli bir dizi (`char c[2] = "ab"`) ile denedi ve kabul edilmediğini gösterdi. Değerin tam sayı ya da karakter olması gerekir, ondalıklı sayı da kullanılamaz. `gcc` ile denendi: Dizi ve `float` için "switch quantity not an integer" hatası verdi.

> **Hocanın vurgusu:** Bu ders boyunca `break` yalnız `switch` içinde kullanılır. Başka bir yerde `break` kullanan çözüm kabul edilmez. Ayrıntı için 18. bölüme bakın.

15. while ve do-while

`while` döngüsünde koşul doğru olduğu sürece gövde çalışır. Koşul hiçbir zaman doğru olmazsa gövde hiç çalışmaz. Hoca bu ihtimalin önemli olduğunu söyledi.

Hoca bir de farklı bir yazım gösterdi: Koşuldan sonra bir şey yazıp, altında durduk yere açılıp kapanmış bir blok bırakmak. Koşul doğruysa sonsuz döngüye girilir, yanlışsa girilmez ve aradaki anonim blok işletilir.

```c
while (koşul);
{
    /* anonim blok */
}
```

`do-while` döngüsünde şart sonradan sorulur. Hoca önce "en az bir defa yapılma ihtimali var" dedi, sonra kendisi düzeltti: Gövde kesinlikle en az bir defa yapılır. Şart yanlışsa bir defa, doğruysa iki, üç veya daha fazla defa çalışır.

```c
do {
    /* gövde */
} while (koşul);
```

`for` döngüsü daha önce ayrıntılı anlatıldığı için geçildi.

16. Boş İfadeler ve Sık Yapılan Hatalar

16.1 Boş İfade (Null Statement)

Geçen derste `getchar` görülmüştü. Bir `scanf` ile sayı okuduktan sonra `getchar` ile karakter okuyunca, önceki satırda girilen Enter tuşu saklı kaldığı için okunan karakter o olur. Döngülerde buna dikkat etmek gerekir. Hoca gövdesi boş olan şu döngüyü gösterdi:

```c
#include <stdio.h>

int main(void) {
    int c;
    while ((c = getchar()) == ' ');
    printf("%d\n", c);
    return 0;
}
```

Döngü bir karakter okur. Bu karakter boşluksa tekrar okur. Hiç gövdesi yoktur, işi koşulun içinde yapar. Amaç, düzgün bir karakter girilene kadar okumaktır. Hoca çalıştırdı, Enter, boşluk ve Tab tuşlarına bastı, bunların hepsini "boşluk" diye niteledi. `T` tuşuna basınca döngüden çıkıldı.

> **Ders sonrası not:** Hoca Enter ve Tab için de döngünün devam ettiğini anlattı, ama `== ' '` koşulu yalnız boşluk karakterini kapsar. Yukarıdaki kod `gcc` ile derlenip denendi: Girdi Enter ile başlayınca 10, Tab ile başlayınca 9 kodlu karakterle döngüden çıktı. Bütün boşluk türlerini atlamak için `while (isspace(c = getchar()));` yazılabilir (`ctype.h`). Bu biçim denendi ve `T` karakterine kadar okudu.

Hoca aynı mantığı `for` için de gösterdi:

```c
int i = 1, j = 0;
for (; i < 5; ) {
    j++;
    printf("%d\n", j);
    i++;
}
```

Atama kısmı boştur, arttırma kısmı da boştur. Bu kod çalışır, çünkü başlangıç değerini ve arttırmayı yazma zorunluluğu yoktur, şarta müdahale gövdede yapılmıştır. `gcc` ile çalıştırıldı, 1, 2, 3, 4 yazdırdı. Hoca bunu, geçen derste konuşulan dizi erişimindeki farklı yazım gibi bir "programlama hilesi" olarak nitelendirdi.

> **Hocanın vurgusu:** Bu yazımlar doğru kullanım değildir. Programlama hilesi yapmıyoruz. Bir problemi çözen ve herkes tarafından anlaşılan program yazıyoruz. Başlangıç değeri ve arttırma `for`'un kendi yerlerine yazılmalıdır.

16.2 if'ten Sonra Noktalı Virgül

```c
if (c == 1);
j = 0;
```

Noktalı virgül `if`'in gövdesini boşaltır. `c` 1'e eşitse hiçbir şey yapılmaz. Eşit olmasa da bir sonraki satır çalışır, yani `j = 0;` her durumda işletilir.

16.3 Karşılaştırma Yerine Atama

```c
if (j = 0) { /* ... */ }
while (i = 5) { /* ... */ }
```

Soru: Bu iki satır ne yapar? **Cevap:** İkisi de karşılaştırma değil atamadır. İlkinde `j`'ye 0 atanır, ifadenin değeri 0 yani yanlıştır, gövde hiç çalışmaz. İkincisinde `i`'ye 5 atanır, ifadenin değeri 5'tir yani doğrudur, sonsuz döngü oluşur. `gcc` ile denendi: `if (j = 0)` gövdeye girmedi ve `j` 0 oldu. `-Wall` bu satırda parantez uyarısı verdi.

17. İç İçe Döngüler

Hoca iç içe döngülerin bilindiğini varsaydı, ama kısaca çarpım tablosu üzerinden gösterdi. Slaytta üstte başlık satırını yazan parantezsiz bir `for` vardı, bu döngü 1'den 9'a kadar değerleri yazar. Sonra alt satıra geçilir. Ardından `i` ve `j` için 1'den 9'a giden iç içe iki döngü gelir. Slaytın anlatıma göre yeniden yazımı aşağıdadır, `gcc -Wall` ile derlendi.

```c
#include <stdio.h>

int main(void) {
    int i, j;

    for (j = 1; j <= 9; j++)
        printf("%4d", j);
    printf("\n");

    for (i = 1; i <= 9; i++) {
        for (j = 1; j <= 9; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

Çalışma sırası şöyledir:

1. `i` = 1, `j` = 1 için gövde çalışır.
2. `i` = 1 kalır, `j` 2, 3, …, 9 olur ve gövde her biri için çalışır.
3. `j` 10 olunca iç döngü tekrar etmez, biter.
4. Dış döngü ilerler: `i` = 2 olur ve `j` yeniden 1'den başlar.
5. Bu işlem `i` 9 olana kadar sürer.

> **Hocanın vurgusu:** En içteki döngü tamamlanmadan dıştaki döngüye sıra gelmez.

18. break, continue ve goto

Bu ifadeler programlama dillerinde vardır ve yerine göre gerekebilir. Fakat algoritma tasarımını ihlal ettikleri ve tasarım kurallarına aykırı oldukları için bu derste kullanılmazlar.

> **Hocanın vurgusu:** `switch` içindeki `break` dışında hiçbir yerde `break`, `continue` ve `goto` kullanılmayacak. Bu ifadeleri içeren çözümler istenmiyor.

19. Ön İşlemci: Makrolar

Ön işlemcide yeni konu makrolardır. Makro bir yer değiştirmedir (substitution).

19.1 Excel Benzetmesi

Hoca makroyu anlatmak için sınıfa Excel'de makro kullanıp kullanmadıklarını sordu ve canlı bir örnek yaptı. Bir görevi kaydediyorsunuz, sonra çalıştırdığınızda kaydedilen bütün adımlar sizin için yapılıyor. Yaptığı adımlar şunlardır:

1. Sayfaya hücreler ve bir buton ekledi.
2. Makro kaydını başlattı, kaydın adı `Makro1` oldu.
3. Veriyi seçip kopyaladı, ikinci sayfaya gidip yapıştırdı ve alta bir satır ekledi.
4. Kaydı bitirip makroyu butona atadı.
5. Yeni değerler yazıp butona bastı, aynı işlemler kendiliğinden yapıldı.
6. Makroları görüntüleyip düzenleme ekranını açtı. Kayıttaki adımların arka planda başka bir dilde kod olarak yazıldığını gösterdi: aralık seçme, kopyalama, başka sayfaya geçme, yapıştırma.

Makro bir anlamda fonksiyon gibidir, ama farkları vardır. C'de de benzerini yapabiliriz.

19.2 Makro Tanımlama

Makrolar ön işlemci komutlarıdır, `#define MAX 10` yazmak gibidir. Tanımın yapısı şöyledir:

- Başta `#define` gelir.

- Sonra makronun adı yazılır. Hoca adın, minimum değer için `MIN_VALUE` gibi iki kelimeli ve eylem bildiren bir ifade olmasının iyi olacağını söyledi.
- Bir boşluk bırakılır ve makronun karşılığı olan ifade yazılır.
- İfade alt satıra taşacaksa satır sonuna `\` konarak devam edilir.
- Sonuna noktalı virgül konmaz.

En basit makro, sürekli kullanılacak bir stringtir. Hoca bir menü metniyle gösterdi:

```
#define MENU_YAZ "Hosgeldiniz\n1-Toplama\n2-Cikarma\
\n3-Carpma"
```

Bu ifade çağrılmaz, kendisi gelir. Normal bir fonksiyon çağrıldığında programın akışı, hocanın örneğindeki gibi yedinci satırdan onuncu satıra geçer. Makroda ise daha derleme aşamasındayken makronun adı kaybolur ve yazıldığı yere karşılığı kopyalanır. Kod genişletilmiş olur (code expansion).

Soru: Makronun fonksiyona göre ne kolaylığı var? **Cevap:** En temel fark, fonksiyona dallanma (branch) yapılması ve bunun yığın (stack) belleğini fazladan kullanmasıdır. Makroda böyle bir şey yoktur, program bellekte çalışma aşamasına gelmeden kod yerine yapıştırılmıştır. Çok sık kullanılan bir pi sayısı ya da parametreli işlemler için makro yazmak programı daha rahat yazılır ve kısmen daha hızlı hale getirir.

Bir başka örnek tampon uzunluğudur. Burada parantez kullanılmıştır ve bu sorun değildir, derleme aşamasında olduğu gibi kopyalanır ve 512 uzunluğunda bir karakter dizisi oluşur:

```
#define BUF_LEN (512)
char tampon[BUF_LEN];
```

19.3 Fonksiyon Benzeri Makrolar ve Parantez Sorunu

Makro bir veya iki parametre de alabilir. Hoca, bir sayıyı 2 ile çarpmanın kendisiyle toplamak olduğunu, çarpmanın işlem süresi açısından daha maliyetli olduğunu, mikroişlemci seviyesinde `a + a`'nın daha hızlı olacağını söyledi ve makroyu buna göre yazdı. Çağırırken noktalı virgül kullanılır, çünkü o bir atama ifadesidir. Tanımda ise noktalı virgül kullanılmaz.

```
#define MULT_BY_TWO(a) a+a

int x = MULT_BY_TWO(5);
```

> **Ders sonrası not:** Hoca `a + a`'nın `2 * a`'dan hızlı olduğunu söyledi. `gcc` ile `-O0` ve `-O2` seçenekleriyle denendi: İki yazım bu makinede aynı Assembly kodunu üretti. Bu durumda elle dönüştürmenin hız kazancı yoktur.

Dersteki denemeler aşağıdadır. Sonuçlar `gcc` ile derlenip çalıştırılarak doğrulandı.

Çağrı	Genişleme	Sonuç
`MULT_BY_TWO(5)`	`5+5`	10
`MULT_BY_TWO(-3)`	`-3+-3`	-6
`MULT_BY_TWO(2*3)`	`2*3+2*3`	12
`MULT_BY_TWO(2+3)`	`2+3+2+3`	10

Burada sıkıntı görülmez, çünkü çarpma toplamadan önceliklidir ve toplamalar kendi aralarında aynı sonucu verir. Hoca ayrıca makroya `2_3` gibi geçersiz bir sembol vermeyi denedi. Derleyici bunu kabul etmedi ve tam sayı sabiti beklediğini belirtti. `gcc` ile denendi: "invalid suffix" hatası verdi.

Hoca asıl sorunu kare makrosuyla gösterdi:

```
#define SQR(x) x*x
```

Çağrı	Genişleme	Sonuç
`SQR(2*3)`	`2*3*2*3`	36
`SQR(2+3)`	`2+3*2+3`	11

Çağrı	Genişleme	Sonuç
`SQR((2+3))`	`(2+3)*(2+3)`	25

Sınıfın beklentisi `SQR(2+3)` için 25'ti. Oysa kod `2 + 3 * 2 + 3` haline gelir, önce çarpma yapılır: $2 + 6 + 3 = 11$.

> **Ders sonrası not:** Hoca `SQR(2+3)` için önce 13 dedi. 01:45:35 civarında sınıftan gelen uyarıyla düzeltti ve 11 olduğunu söyledi. Doğrusu 11'dir (`gcc` ile çalıştırıldı).

Bunun üstesinden gelmek için tanımda her parametre paranteze alınır:

```
#define SQR1(x) (x)*(x)
```

Artık `SQR1(2+3)`, `(2+3)*(2+3)` olur ve 25 verir. Hoca örnekleri bilerek aşamalı yaptığını söyledi. `MULT_BY_TWO` tanımında operatör toplama olduğu için sorun çıkmamıştı, ama normalde orada da parantez kullanmak gerekir.

> **Hocanın vurgusu:** Makro tanımlarken parantez kullanmak önemlidir. Her parametre ayrı ayrı paranteze alınır.

> **Ek bilgi (derste anlatılmadı):** Makronun tamamı da paranteze alınmazsa büyük olasılıkla çevresindeki operatörlerle karışır. Kısa nottaki örnek: `3 * MULT_BY_TWO(2)` ifadesi 3*2+2 olur ve 8 verir (`gcc` ile çalıştırıldı). Argüman makroda iki kez geçiyorsa yan etkisi de iki kez çalışır: Kısa nottaki `int a = 2; int r = SQR1(a++);` satırı `(a++)*(a++)` olur. Bu büyük olasılıkla tanımsız davranıştır, `gcc -Wall` uyarı verdi ve bu makinede `r` = 6, `a` = 4 çıktı.

19.4 En Küçüğü Bulan Makro

Hoca koşul operatörünü kullanarak iki sayıdan küçüğünü bulan bir makro yazdı. Eşitlik durumunda ikisi de aynı olacağı için `<=` kullandı. Parametreler karmaşık ifadeler olabileceği için her biri ayrı ayrı, ifadenin tamamı da ayrıca paranteze alındı:

```
#define MIN(x,y) ((x)<=(y)?(x):(y))
```

Üç sayı için yeni bir tanım yazılabileceğini söyledi ve bunu çalışma sorusu olarak bıraktı. Sonra daha kısa bir yol gösterdi: Üçlü makro yazmaya gerek yoktur, ikili makro iç içe kullanılabilir. Dört sayı için de aynısı geçerlidir:

```
MIN(MIN(2,6), 7)          /* 2 */
MIN(MIN(2,6), MIN(3,8))   /* 2 */
```

Adım adım: `MIN(2,6)` 2 verir, `MIN(3,8)` 3 verir, `MIN(2,3)` 2 verir. Sonuç `gcc` ile doğrulandı ve derste de 2 geldi.

Derste aynı isimle iki tanım (`MIN(x,y)` ve `MIN(x,y,z)`) yazılınca derleyici argüman sayısıyla ilgili hata verdi. Hoca üç parametreli tanımı silip farklı bir ad vermeyi önerdi.

> **Ders sonrası not:** C'de makrolar aşırı yüklenemez, aynı ismin ikinci tanımı öncekinin yerine geçer. `gcc` ile denendi: "MIN redefined" uyarısı verdi. Üç sayı için ayrı isim kullanılır. Çalışma sorusunun kısa nottaki cevabı `#define MIN3(x,y,z) MIN(MIN(x,y),z)` şeklindedir, `MIN3(3,1,7)` `gcc` ile 1 çıktı.

19.5 Genişletilmiş Kodu Görmek

Hoca ilk haftaki uygulamayı hatırlattı: `gcc`'ye `-save-temps` seçeneği verilince ara dosyalar saklanıyordu. Bu seçenekle kodun nasıl genişlediğini göstermek istedi, ama bilgisayarında `gcc` yol tanımlı (PATH) olmadığı için terminalde çalıştıramadı ve denemeyi öğrencilere bıraktı.

Dört sayılı `MIN` satırı `gcc -E -P` ile genişletildiğinde aşağıdaki çıktı alınır (okunsun diye satırlara bölündü):

```
((((2)<=(6)?(2):(6)))<=(((3)<=(8)?(3):(8)))
    ?(((2)<=(6)?(2):(6)))
    :(((3)<=(8)?(3):(8))))
```

> **Ek bilgi (derste anlatılmadı):** Bu makinede `gcc -save-temps -c` komutu kaynak dosyanın yanına `.i`, `.s` ve `.o` uzantılı dosyalar bıraktı. `.i` dosyası ön işlemcinin çıktısı, yani makroların genişletilmiş halini içerir. Hocanın öğrencilere bıraktığı inceleme bu dosyada yapılabilir.

Hoca özetle şunu tekrarladı: Makro tamamen bir yer değiştirmedir, fonksiyon dallanması yoktur (no jump). Yan etkilere dikkat etmek gerekir. Örneğin `MIN`'e yan etkili bir ifade gönderilirse ne olacağına bakmayı öğrencilere bıraktı.

20. Makro ile Fonksiyonun Karşılaştırılması

Hoca makronun avantajlarını ve dezavantajlarını şöyle sıraladı:

- **Avantajları:**
  - Fonksiyon çağrısından gelen ek yük (overhead) yoktur, çünkü fonksiyon çağrılmaz.
  - Tip kısıtlaması yoktur, aynı makro birkaç veri türünü destekleyebilir.
- **Dezavantajları:**
  - Her kullanımda kod tekrar yapıştırılır, kod uzayabilir.
  - Yan etkilere yol açabilir.
  - Hata ayıklamayı zorlaştırabilir.

Hata ayıklama konusunu derste gösterdi. Normalde bir satıra kesme noktası (breakpoint) koyup debug yapılabilir. Fakat program o satıra geldiğinde makro diye bir şey kalmamıştır, yerine kodu yapıştırılmıştır. Bu yüzden makronun doğru çalışıp çalışmadığını test etmek zorlaşır.

21. Makroyu Kaldırmak ve Hazır Makrolar

21.1 #undef

Bir makro tanımı kaldırılabilir. Hoca slayttaki örneği anlattı: `FALSE` 1 olarak tanımlanmış ve kod üzerinde işlem yapılıyor. Ardından `#undef` ile geçersiz kılınıyor ve bu satırdan itibaren `FALSE` 0 olarak tanımlanıyor.

```
#define FALSE 1
/* ... */
#undef FALSE
#define FALSE 0
```

`gcc -Wall` ile derlendi, uyarı vermedi. `FALSE` değeri ilk bölümde 1, sonrasında 0 çıktı.

21.2 Hazır Makrolar

Bazı makrolar hazır gelir (built-in): `__LINE__` kaynak kodda kaçıncı satırda olduğunuzu, `__FILE__` hangi dosyada olduğunuzu, `__TIME__` saati, `__DATE__` tarihi verir. `__STDC__` ise derleyici ANSI C standardına uygunsa 1 verir. Hoca `__LINE__`'ı `%d` ile yazdırdı ve 28 çıktı, yani bulunduğu satırın numarası.

Soru: Bu makrolar nerededir, biz yazmadık? Sınıftan "standart girdi çıktı olabilir mi?" önerisi geldi. **Cevap:** Hoca, genellikle `stdio.h` içinde bulunduklarını söyledi.

> **Ders sonrası not:** Hoca bu makroların `stdio.h` içinde olduğunu söyledi; doğrusu büyük olasılıkla bunların hiçbir başlık dosyasında olmadığı, derleyicinin ön işlemcisi tarafından önceden tanımlandığıdır. `gcc` ile denendi: `#include` olmayan bir dosyada `__LINE__` derlendi ve çalıştı.

Soru: Kodun kaçıncı satırında olduğunu bilmek çalışan bir programda neden önemlidir? **Cevap:** Kodun ne zaman derlendiği ve kaçıncı satırına ulaşıldığı loglanabilir, bu da bazı açılardan hata ayıklamayı mümkün kılar. Programın bir noktasına böyle bir mesaj koyarsınız. Mesajı görüyorsanız kod oraya kadar sorunsuz çalışmıştır. Mesaj yerine başka şeyler görüyorsanız sorun yukarıda bir yerdedir.

Bu makroların tablosu ders notlarındadır.

22. Koşullu Derleme

Koşullu derleme (conditional compilation), ön işlemci komutlarının `#if`, `#else`, `#elif`, `#endif` gibi yapılarla koşula bağlı çalıştırılmasıdır. Bu koşullar derleme sırasında değerlendirilir, derlemeden sonra karşınıza gelmez.

Hoca kullanım alanı olarak işletim sistemini örnek verdi: İşletim sistemi Windows ise koda Windows'a göre, Linux ise Linux'a göre derlemek isteyebilirsiniz. Benzer şekilde derleyiciye veya `gcc`'nin komut satırı seçeneklerine göre de koşul kurulabilir.

Slayttaki basit örnek, sonradan kullanılacak `x` ve `y`'nin değerini belirliyordu. Anlatıma göre yeniden yazılan biçimi şudur:

```
#if x == 1
    #undef x
    #define x 0
#elif x == 2
    #undef x
    #define x 3
#else
    #define y 4
#endif
```

`x` 1 ise önceki değeri kaldırılır ve 0 yapılır. `x` 2 ise önceki değeri kaldırılır ve 3 yapılır. Hiçbiri değilse `y` 4 olarak tanımlanır. `gcc -Wall` ile `#define x 2` başlangıcıyla denendi ve `x` 3 çıktı.

Bir makronun tanımlı olup olmadığına da bakılabilir:

```
#if defined(TEST)
    #undef TEST
#endif
```

`TEST` adında bir makro tanımlanmışsa bu örnek onu kaldırır. Tanımlıysa bir işi, değilse başka bir işi yaptırmak da mümkündür. `#ifdef TEST` ve `#ifndef TEST` bu denetimin kısa yazımıdır:

```
#ifdef TEST
    /* TEST tanımlıysa */
#else
    /* TEST tanımlı değilse */
#endif
```

`#include` ise daha önce anlatılmıştı, açılı parantezli ve tırnaklı kullanım farkı konuşulmuştu. Satır denetimi (`#line`) konusu geçildi. Koşullu derlemeyle ilgili birkaç örnek de sonraki derse bırakıldı.

Dersin Toparlaması

Bu derste önce `typedef` ile var olan tiplere, özellikle yapılara kısa isim vermeyi gördük ve prototiplerin yapı tanımından sonra yazılması gerektiğini öğrendik. Ardından ifade türlerini ve operatörleri hızlıca tekrarladık: virgül, kalan, atamalı işlemler, önek ve sonek arttırma, ilişkisel ve mantıksal operatörler, kısa devre, bit işlemleri, koşul operatörü ve bellek operatörleri. Kontrol akışında süslü parantezin önemini, `for` döngüsünün çalışma sırasını, karşılaştırma sayısını, `switch` içindeki `break` etkisini, `while` ile `do-while` farkını ve atama ile karşılaştırma karışıklığını ele aldık.

Dersin yeni konusu makrolardı. Makronun derlemeden önce yapılan bir metin yer değiştirmesi olduğunu, parametrelerin neden paranteze alınması gerektiğini, fonksiyona göre artılarını ve eksilerini gördük. `#undef`, hazır makrolar ve koşullu derleme ile bölümü bitirdik. Sonraki derste koşullu derlemeyle ilgili örneklerle devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.